

Predicting Urban Micromobility Trip Distance Using Linear and Neural Network Models

Bryant Willoughby
Master's Student in Applied Statistics
University of Michigan
Ann Arbor, MI

Abstract—This study models urban micromobility trip distance using a large real-world dataset from Austin, Texas, comparing linear regression, single-layer perceptrons, and multi-layer perceptrons (MLPs). The response is log-transformed, and predictors are standardized for stable optimization. Results show that linear models achieve limited accuracy, while a shallow ReLU-activated MLP significantly improves performance. A final GPU-based mini-batch gradient descent (MBGD) model further enhances predictive accuracy, demonstrating the value of stochastic training for nonlinear mobility modeling.

Index Terms—Micromobility, neural networks, regression, deep learning, transportation analytics

I. INTRODUCTION

Shared micromobility systems, including bicycles, e-bikes, and electric scooters, have become integral components of urban transportation networks. These services provide flexible, short-distance mobility and often complement existing public transit by facilitating efficient first- and last-mile travel. As adoption continues to grow, cities increasingly rely on data-driven approaches to understand usage patterns and support planning, deployment, and regulatory decisions.

Trip distance is an especially important behavioral metric, reflecting how riders use micromobility infrastructure across temporal and operational contexts. Its determinants—such as trip duration, time of day, vehicle type, and local geography—can interact nonlinearly, which limits the expressiveness of classical linear models. This motivates the use of more flexible learning approaches capable of capturing richer structure in the data.

The objective of this study is to predict the log-transformed distance of micromobility trips using operational and temporal features from a large real-world dataset. The modeling framework compares multiple linear regression, a single-layer perceptron (SLP), and several multi-layer perceptron (MLP) architectures. A final GPU-accelerated mini-batch training regime is also evaluated to examine how modern optimization strategies affect model performance and generalization.

A. Related Work

Machine learning has shifted from symbolic reasoning to predictive modeling driven by large datasets and computational advances [1]. Deep learning expanded this progress by enabling multi-layer architectures capable of representing highly nonlinear patterns [2]. Foundational work on backpropagation

[3] and universal approximation [4] provides theoretical justification for using shallow MLPs in regression tasks.

Neural networks have been widely applied to transportation research, including traffic-flow prediction [5] and city-scale ridership modeling [6]. These studies highlight the value of nonlinear models for capturing behavioral patterns in human mobility, motivating the present work.

B. Data Description

The dataset originates from Austin's Open Data Portal [7] and spans over 15 million micromobility trips from April 2018 to April 2022. Each record provides start and end timestamps, reported distance, duration, vehicle type, and council district identifiers.

To avoid structural shifts caused by later operational changes or the COVID-19 pandemic, we restrict analysis to weekday trips during 2018, yielding roughly 1.67 million observations.

Numerical fields containing non-numeric characters were parsed using regular expressions. Observations with non-positive distance or duration were removed, as were redundant identifiers. Missing categorical entries were imputed via mode. Timestamps were mapped to Unix time, and the response was log-transformed to mitigate right skew.

The log-transformed origin–destination matrix (Figure 1) shows strong spatial variation, with District 9 having the highest trip volume and districts 1, 3, and 5 forming secondary mid-volume connections.

Ten predictors describing temporal, operational, and contextual attributes were used (Table I).

TABLE I
VARIABLES USED IN MODELING

Type	Name	Description
Response	Log Trip Distance	Natural log of distance (miles)
Predictor	Trip Duration	Seconds
Predictor	Month	1–12
Predictor	Hour	0–23
Predictor	Day of Week	0=Mon,...,6=Sun
Predictor	Start District	Council district
Predictor	End District	Council district
Predictor	Year	2018 indicator
Predictor	veh_scooter	Scooter indicator
Predictor	veh_bicycle	Bicycle indicator
Predictor	start_ts	Unix start time
Predictor	end_ts	Unix end time

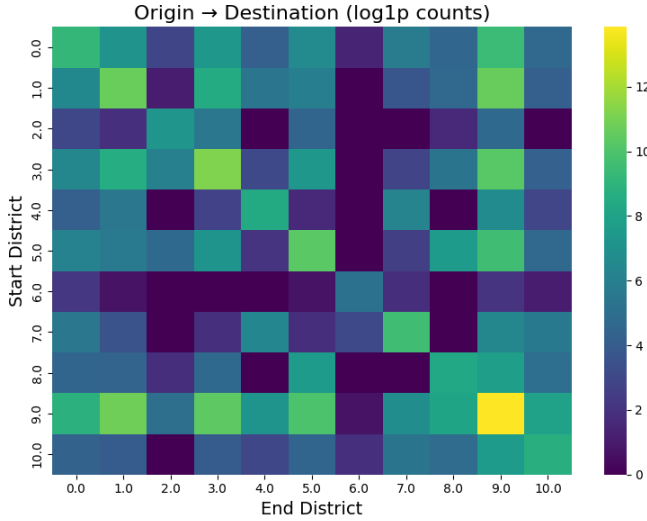


Fig. 1. Origin → Destination heatmap (log1p counts).

II. METHODS

The modeling task is formulated as the estimation of the conditional mean of the log-transformed trip distance. For each trip i , let $x_i \in \mathbb{R}^p$ denote the vector of predictors and let y_i denote the corresponding log-distance. All models considered here seek to learn a parametric function $f(x_i; \theta)$ that minimizes the empirical risk

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta))^2,$$

where θ denotes all learnable parameters. Model performance is assessed using a held-out validation set during training and a separate test set for final evaluation.

A. Data Splitting and Standardization

After preprocessing, the dataset was partitioned into training (70%), validation (15%), and test (15%) sets. To prevent information leakage, standardization parameters—means and standard deviations of the predictors—were computed strictly from the training data and subsequently applied to the validation and test sets. This ensures that all downstream model estimates and predictions rely only on information available at training time.

B. Multiple Linear Regression

Multiple linear regression (MLR) serves as a transparent baseline model. The standardized predictors are arranged into a design matrix X , and the model assumes a linear relationship

$$\hat{y} = X\beta,$$

where β is estimated via ordinary least squares. Because standardization places all predictors on a common scale, coefficient magnitudes provide direct insight into feature importance. However, the linear form limits the model’s ability to capture interactions or nonlinear effects.

C. Single-Layer Perceptron

The single-layer perceptron (SLP) retains the linear structure of MLR but replaces closed-form estimation with gradient-based optimization. The model computes

$$\hat{y} = XW^\top + b,$$

where W and b are optimized using the Adam algorithm. This formulation allows the SLP to share optimization dynamics with deeper networks, isolating the effect of model capacity from the effect of gradient-based training. Early stopping on the validation loss prevents unnecessary computation and reduces the risk of overfitting.

D. Multi-Layer Perceptron

The multi-layer perceptron (MLP) extends the SLP by introducing one or more hidden layers and nonlinear activation functions. A one-hidden-layer MLP computes

$$h = \phi(XW_1^\top + b_1), \quad \hat{y} = hW_2^\top + b_2,$$

where ϕ is a nonlinear activation function such as ReLU. Deeper MLPs stack multiple layers of this form, allowing the network to represent complex, high-order interactions. During training, the validation loss is monitored at each epoch, and training halts when improvement falls below a predefined tolerance for several consecutive epochs.

The MLP experiments systematically vary activation function choice, network depth, learning rate, and dropout probability. This enables a structured evaluation of how architectural and optimization choices affect convergence, generalization, and computational efficiency.

E. Mini-Batch Gradient Descent and GPU-Accelerated Training

Because full-batch gradient descent produces only one update per epoch, it can converge slowly and may become trapped in sharp minima. To address this, the final model was additionally trained using mini-batch gradient descent (MBGD). At each iteration t , the gradient is approximated by a batch B_t :

$$\nabla_{\theta} \mathcal{L}_{B_t}(\theta) = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \ell(f(x_i; \theta), y_i).$$

This introduces controlled stochasticity, leading to more frequent parameter updates and potentially better generalization.

Training was implemented using PyTorch’s `DataLoader` with batch size 8192 and performed on an NVIDIA CUDA GPU when available. All tensors were moved to the GPU, and forward- and backward-pass computations were executed on-device. Random seeds were fixed for Python, NumPy, and PyTorch (CPU and CUDA) to ensure reproducibility. Early stopping used patience 5 and a relaxed tolerance $\text{min_delta} = 10^{-3}$ to accommodate the noisier validation trajectory typical of MBGD.

III. METHODS

The goal of the modeling framework is to approximate the conditional mean of the log-transformed trip distance using a parametric function of the observed predictors. For each trip i , let $x_i \in \mathbb{R}^p$ denote the vector of predictors and let y_i denote the corresponding log-distance. All models considered here learn a function $f(x_i; \theta)$ that minimizes the empirical risk

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta))^2,$$

with model parameters θ estimated from the training set and evaluated using a held-out validation and test set.

A. Data Splitting and Standardization

After preprocessing, the dataset was partitioned into training (70%), validation (15%), and test (15%) sets. To prevent information leakage, standardization parameters—means and standard deviations of the predictors—were computed strictly from the training data and applied to the validation and test sets. This ensures that downstream predictions rely only on information available at training time.

B. Multiple Linear Regression

Multiple linear regression (MLR) serves as a transparent baseline. The standardized predictors form a design matrix X , and the model assumes

$$\hat{y} = X\beta,$$

where β is estimated via ordinary least squares. Because predictors are standardized, coefficient magnitudes are directly comparable; however, the linear form limits the model's capacity to capture nonlinear or interaction effects.

C. Single-Layer Perceptron

The single-layer perceptron (SLP) preserves the linear structure of MLR but replaces closed-form estimation with gradient-based optimization. It computes

$$\hat{y} = XW^\top + b,$$

where parameters (W, b) are optimized using the Adam algorithm. Since the SLP is linear in its parameters, it isolates the effect of optimization dynamics rather than model capacity.

Early stopping is used to halt training when validation loss fails to improve beyond a fixed tolerance. The SLP training and validation loss curves are shown in Figure 2.

D. Multi-Layer Perceptron

The multi-layer perceptron (MLP) extends the SLP by introducing one or more hidden layers and nonlinear activation functions. A one-hidden-layer MLP computes

$$h = \phi(XW_1^\top + b_1), \quad \hat{y} = hW_2^\top + b_2,$$

where ϕ is a nonlinear activation such as ReLU. Deeper architectures stack multiple hidden layers, allowing the network to represent complex patterns and high-order interactions.

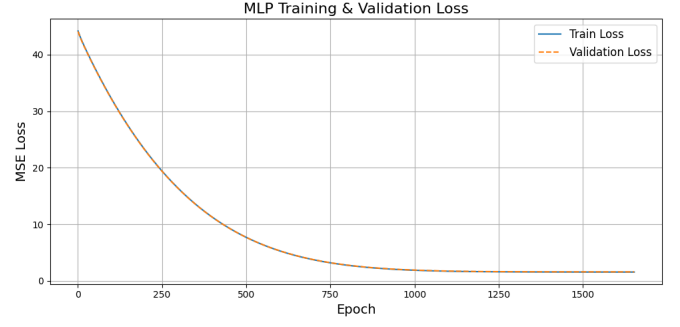


Fig. 2. Training and validation loss for the SLP (full-batch training).

Training proceeds via gradient descent using Adam. At each epoch, validation loss is computed and monitored, and early stopping halts training once improvement drops below a predefined tolerance across several consecutive epochs.

The training and validation curves for the selected MLP architecture are provided in Figure 3.

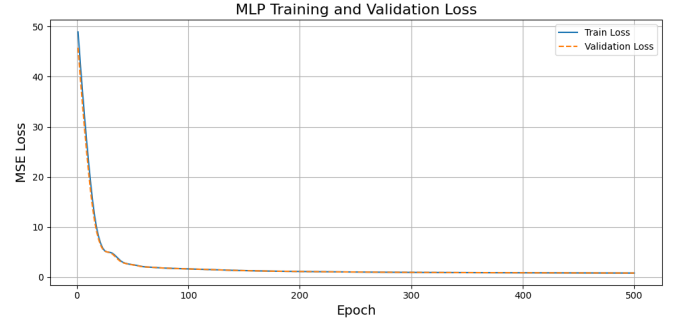


Fig. 3. Training and validation loss for the best-performing MLP (full-batch training).

E. Mini-Batch Gradient Descent and GPU-Accelerated Training

While all hyperparameter sweeps were performed using full-batch gradient descent, the final model was additionally trained using mini-batch gradient descent (MBGD), which replaces the full gradient

$$\nabla_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \ell(f(x_i; \theta), y_i)$$

with a stochastic approximation based on a batch B_t at iteration t :

$$\nabla_{\theta} \mathcal{L}_{B_t}(\theta) = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \ell(f(x_i; \theta), y_i).$$

MBGD performs multiple updates per epoch, injecting controlled stochasticity that promotes better exploration of the loss landscape and reduces the tendency of full-batch methods to converge to sharp minima with poorer generalization.

For this study, batches of size 8192 were used together with the Adam optimizer at learning rate 0.01. Early stopping

employed patience 5 and tolerance `min_delta` = 10^{-3} , a slightly relaxed condition reflecting the noisier validation trajectory inherent in MBGD. Reproducibility was ensured by fixing random seeds for Python, NumPy, PyTorch CPU, and PyTorch CUDA.

To enable GPU acceleration, all tensors were transferred to an NVIDIA CUDA device when available, and PyTorch’s `DataLoader` supplied mini-batches directly to the GPU. Forward and backward passes were executed on-device, allowing matrix multiplications and nonlinear operations to benefit from hardware-level parallelism. Although the MLP architecture is relatively small (one hidden layer, 64 units), MBGD provides a training framework aligned with modern deep-learning practices and scales naturally to larger models or datasets.

IV. CONCLUSION

This study compared linear regression, single-layer perceptrons, and multi-layer neural networks for predicting micromobility trip distance in Austin. Nonlinear models substantially outperformed linear ones, with a simple ReLU-activated MLP showing strong performance and a mini-batch GPU-trained version achieving the best generalization. These findings reinforce the importance of nonlinear modeling and modern optimization methods for mobility analytics.

Preprocessing choices such as log transformation and standardization proved essential for stable training. Hyperparameter experiments highlighted the importance of activation function and learning rate, whereas additional depth and dropout provided limited benefit. Although predictive accuracy was the main focus, interpretability remains relevant; SHAP (SHapley Additive exPlanations) offers a model-agnostic, statistically grounded approach for attributing predictions to individual features and represents a promising extension of this work.

Future research could incorporate more detailed spatial features, explore temporal modeling, or scale to deeper architectures using high-performance computing resources. These extensions would further support data-driven approaches to urban mobility planning and policy.

APPENDIX

All project code and notebooks are available at:

<https://github.com/bryant-willoughby/>

ST-507-Data-Science-Analytics-Using-Python.

REFERENCES

- [1] M. I. Jordan and T. M. Mitchell, “Machine learning: Trends, perspectives, and prospects,” *Science*, vol. 349, no. 6245, pp. 255–260, 2015.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [4] K. Hornik, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [5] Y. Lv, Y. Duan, W. Kang, Z. Li, and F.-Y. Wang, “Traffic flow prediction with big data: A deep learning approach,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 2, pp. 865–873, 2015.
- [6] X. Ma, Z. Tao, Y. Wang, H. Yu, and Y. Wang, “Learning traffic as images: a deep convolutional neural network for large-scale transportation network speed prediction,” in *Proceedings of the 2017 Transportation Research Board Annual Meeting*, 2017.
- [7] City of Austin, “Shared micromobility vehicle trips (2018–2022),” 2022, accessed: 2025-11-18. [Online]. Available: https://data.austintexas.gov/Transportation-and-Mobility/Shared-Micromobility-Vehicle-Trips-2018-2022-/7d8e-dm7r/about_data